Docker: A New Era of Virtualization

March 3, 2025

Presented by,
Harikrishnan S
(TKM23MCA-2031)
Master of Computer Application
Department of Computer Applications
TKM College of Engineering Kollam

Guided By,
Dr.Fousia Shamsudeen
Professor
Department of Computer Applications
TKM College of Engineering Kollam

Overview

- Introduction
- What is Containerization?
- What is a Container?
- What is Docker?
- Advantages of Containerization
- Containers vs Virtual Machines
- How to Containerize an Application
- Dockerfile for a Simple Web App

Introduction

- Modern applications need to be **portable, scalable, and efficient**.
- Traditional deployment methods often lead to **dependency issues** and **resource inefficiency**.
- **Containerization** solves these problems by packaging applications and dependencies into isolated environments.
- **Docker** and **Kubernetes** are leading technologies in this space.

What is Containerization?

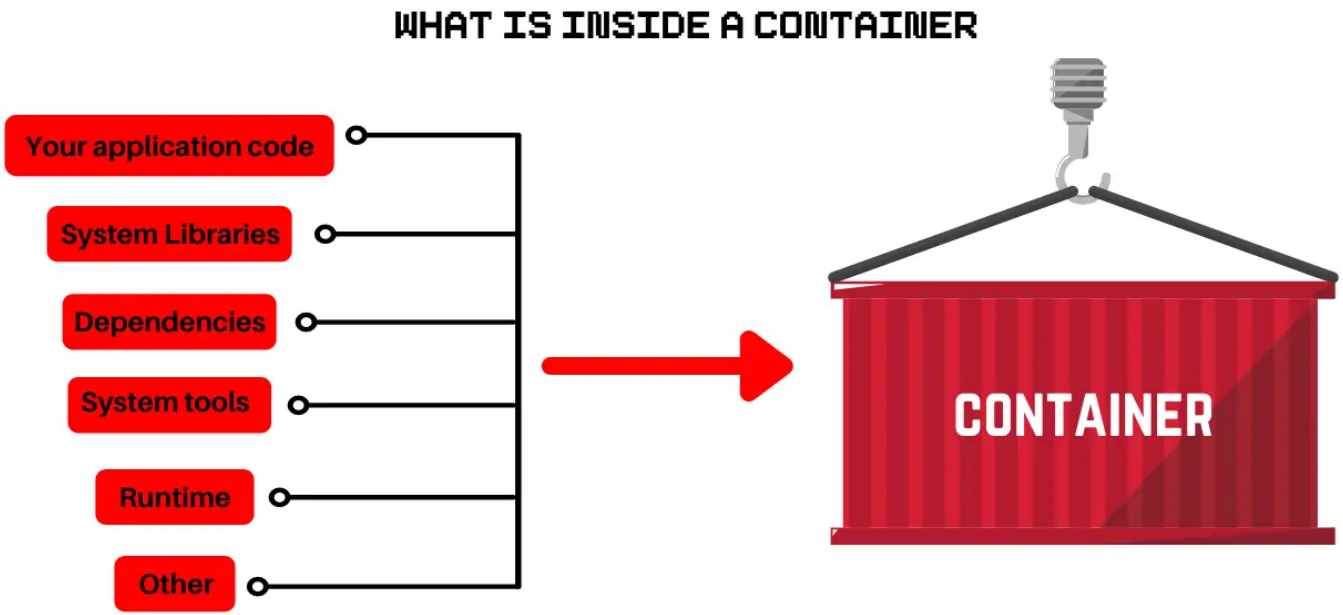
- A method of running applications in **isolated environments**.
- Containers package an application with its **dependencies**, ensuring consistency across different systems.
- Unlike Virtual Machines (VMs), containers **share the host OS kernel**, making them faster and more efficient.

Advantages of Containerization

- **Portability** – Runs the same way on any platform.
- **Efficiency** – Uses fewer resources than VMs.
- **Scalability** – Easily scale applications with Kubernetes.
- **Faster Deployment** – Containers start in seconds.
- **Isolation** – Each container runs separately from others.

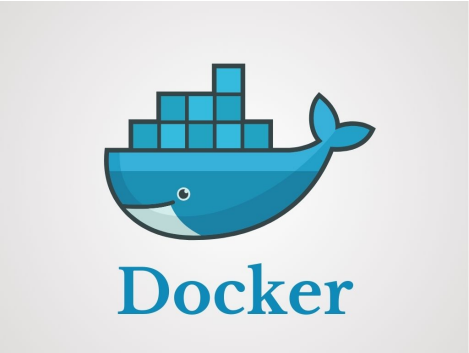
What is a Container?

- A **container** is a lightweight, portable, and isolated environment that includes an application and its dependencies.



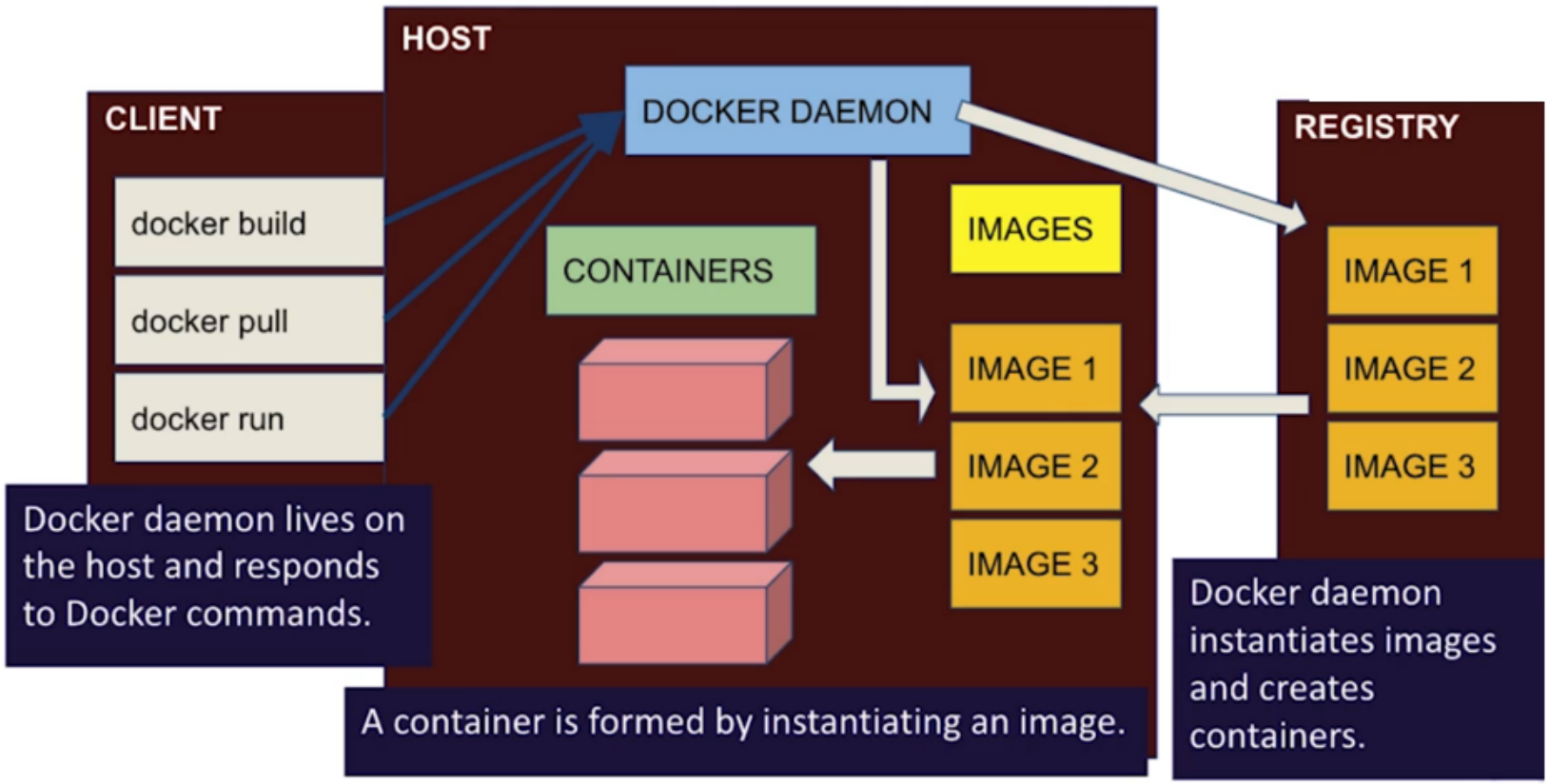
- Containers are managed using tools like Docker and Kubernetes.

What is Docker?

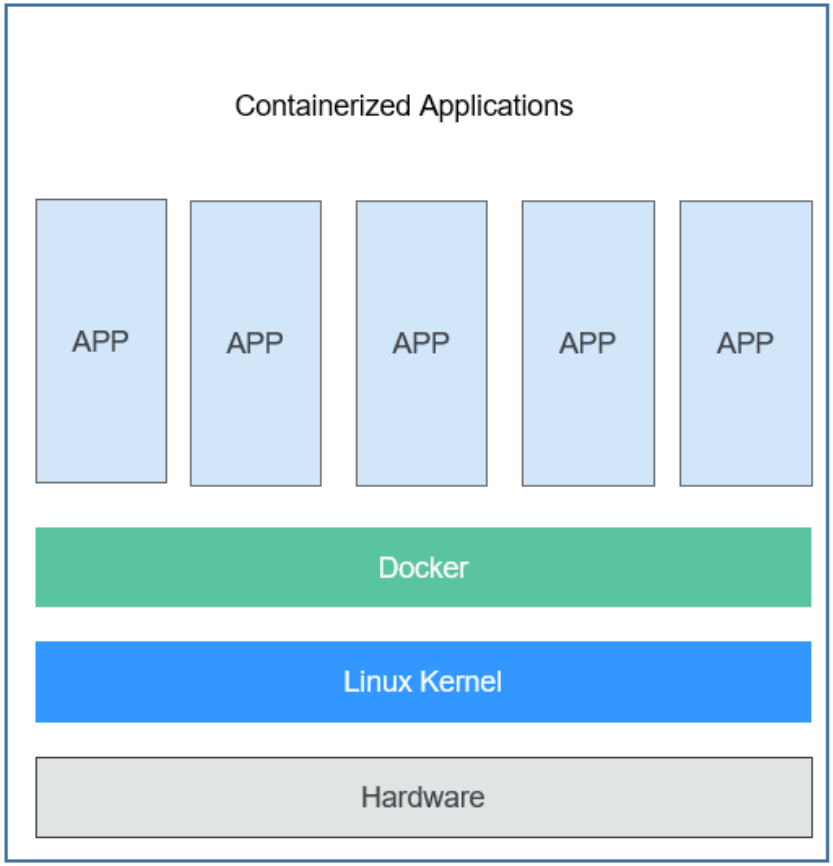


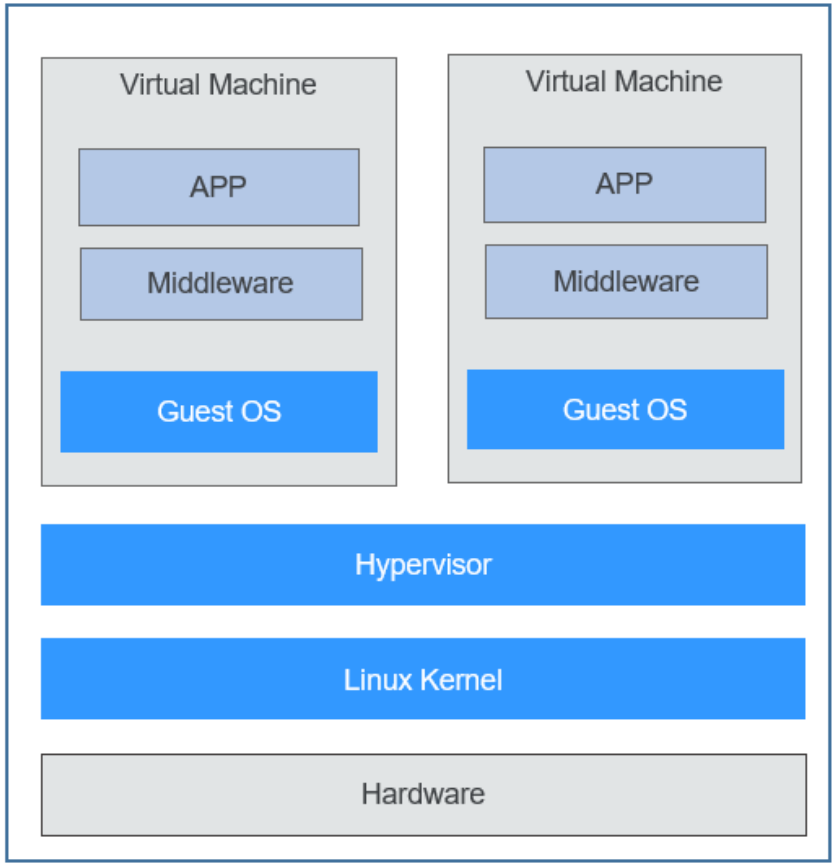
- **Docker** is an open-source platform for **building, shipping, and running** containerized applications.
- It ensures software runs **consistently** across different environments.

Docker Architecture



Containers vs Virtual Machines (VMs)





How to Containerize an Application

- Containerizing an application involves creating a **Docker image**.
- A **Dockerfile** is used to define the container environment.
- Steps to containerize an application:
  - Write a Dockerfile specifying the base image, dependencies, and commands.
  - Build the Docker image using the ‘docker build‘ command.
  - Run the containerized application using ‘docker run‘.

Example: Dockerfile for a Simple Web App

```
FROM python:3.6.9

WORKDIR /app

COPY ./tkmce.py .




ENTRYPOINT [ "python3"]
CMD [ "/app/tkmce.py"]
```

Docker Commands to : Manage Images

Command Name	Syntax	Description
Build Image	`docker build -t <img-name> -f Dockerfile .`	The "**build**" command creates or generates the Image from Dockerfile.
Run Image	`docker run -d <img-name>`	Creates a new container and runs a command or Docker image within a container.
Run Image on Port	`docker run -d -p 8080:8080 <img-name>`	To run a container or image as a container on a specific port the "**-p**" option is used along with the "**run**" command.
List Image	`docker images -a`	List all Docker images.
Tag Image	`docker tag <img-name> <new-img-name>:<tag>`	Tag the Docker image to uniquely identify the image version.
Remove Image	`docker rmi -f <img-name>`	This command is used to remove images forcefully.
Image History	`docker history <img-name>`	This command shows the detailed history of the Docker image.
View Supported Options for Image Build	`docker build --help`	This command shows the supported options for the Docker build.
Inspect Image	`docker inspect image <image-name>`	This will inspect the image in detail

Docker Commands to : Manage Containers

Command Name	Syntax	Description
Create Container	`docker create --name <container-name> -p 5000:5000 <docker-img>`	This command is used to create the Docker container.
List Container	`docker ps -a`	It is used to display all containers.
Start Container	`docker start <container-name>`	This command starts the container. However, users can also use container ID with the "**start**" command.
Stop Containers	`docker stop <container-name>`	The provided command will stop the executing container.
Remove Container	`docker rm <container-name>`	To remove the container, the "**docker rm**" command is used.
Restart Container	`docker restart <container-name>`	This command will restart the docker-stopped container.
Kill Container	`docker kill <container-name>`	It will kill or terminate the running containers only.
Kill All Running Containers	`docker kill $ (docker ps -q)`	This will kill or eliminate all executing containers.

Docker Commands to : Manage Containers

Attach Container	`docker attach <container-name>`	Connect a running container's local input, output, and error streams.
Exposed Port	`docker port <container-name>`	Show the mapping of ports within the container.
Docker container exec	`docker exec -it <container-name>`	The command will open the container shell in which the user can run commands.
Commit Docker Container	`docker commit <image-name>:<tag>`	This will save the container changes in the form of an image. Actually, this command is used to generate images from the container.
Inspect the Container	`docker inspect <container-name>`	This command is used to inspect the Docker container. This will show the network information, host information, and many more.
Container Logs	`docker logs <container-name>`	The command will fetch and display the container's logs.

Docker Commands to : Manage Registry

Command Name	Syntax	Description
Login	`docker login`	This command is used to log in to Docker Hub. Users can also use the "**-u**" option to provide the user name in the command.
Logout	`docker logout`	This command logs out the user from the Docker registry.
Search Image	`docker search <img-name>`	This command is used to search images from the Docker registry.
Push Image	`docker push <img-name>`	This command is used to push the Docker image from the local registry to the remote registry, either in the private or official Docker registry.
Pull Image	`docker pull <img-name>`	This command is utilized for pulling or downloading images from the Docker registry.

Docker Commands to : Manage Volume

Command Name	Syntax	Description
Create Volume	`docker volume create <volume-name>`	This command creates the new volume.
List Volume	`docker volume ls`	List all Docker volumes.
Mount Volume	`docker run [OPTION] --mount source=<volume-name>, target=<container-path> nginx:latest`	This command is used to mount the volume with the Docker container
Remove Volume	`docker volume rm -f <volume-name>`	This command is used to remove volume forcefully.

Docker Commands to : Manage Network

Command Name	Syntax	Description
Create Network	`docker network create <option> <network-name>`	This command creates a new network.
List Network	`docker network ls`	List all available networks.
Inspect Network	`docker network inspect <network>`	This command shows detailed network information.
Connect Network	`docker network connect network container`	This command is utilized to connect the network with the container
Remove Network	`docker network rm <network>`	This command removes the network.

Docker Commands to : Manage Clean

Command Name	Syntax	Description
Docker Prune Volume	`docker volume prune`	This command prunes or removes all unused volumes in Docker.
Docker Prune Image	`docker image prune -a`	This command removes all dangling or unused Docker images.
Docker Prune Container	`docker container prune -a`	The provided command removes all dangling, unused, and stopped containers.
Docker Prune System	`docker system prune`	The specified command completely cleans the Docker by removing all unused, dangling Docker images, networks, and containers. To remove volume along with other components, the "**–volume**" option will be used.
Remove All Containers	`docker rm $(docker ps -aq)`	This command will remove all stopped Docker containers.
Remove All Images	`docker rmi -f $(docker images -aq)`	The provided command will remove all Docker images forcefully.